XR Compression Lab

Viewport-Aware Adaptive Compression for Spatial Audio & Video in Live AR/VR Streaming

Authors: Course Project for Data Compression • **Platform:** Virtual Darshan Synthetic Stereoscopic Rig • **Protocol:** WebRTC (VP8/Opus) & Dual-Plane Media Pipelines

Executive Summary & Core Mission: Immersive Augmented and Virtual Reality (AR/VR/XR) streaming imposes extreme bandwidth and ultra-low latency constraints. Streaming uncompressed 3D stereoscopic video and 360° spatial audio over consumer links causes queueing delays, packet loss, and motion sickness. The **XR Compression Lab** resolves this by combining *classical lossless data compression* (Run-Length, Huffman, LZ77, DEFLATE, LZMA), *transform lossy coding* (8×8 block DCT-II, scalar quantization, Annex-K matrices, libx264/H.264), *4×4 viewport-aware spatial tile allocation* (Chebyshev guard banding, greedy rate-distortion knapsack optimization), *First-Order Ambisonics (FOA) audio down-folding & libopus coding*, and a *closed-loop EWMA bandwidth-adaptive streaming controller*. The system operates across dual planes: a real-time WebRTC browser plane (phone camera rig → aiortc bridge → laptop viewer) and an offline reproducible scientific evaluation harness.

1. Project Capabilities & Architectural Pillars

The XR Compression Lab provides seven interconnected capabilities addressing the spatial media streaming bottleneck:

- **Synthetic & Live Stereoscopic Media Plane:** Generates a calibrated Side-by-Side (SBS) stereoscopic video feed at 1280×720 (left and right eye halves with simulated binocular disparity/parallax over a ~100°×62° frustum) paired with a 4-channel First-Order Ambisonics (FOA) B-format audio field (W omnidirectional, X front-back, Y left-right, Z elevation) simulating a 440 Hz sound source rotating around the listener. Seamlessly swaps to live smartphone or webcam media.
- **Invariant Canvas WebRTC Streaming:** Eliminates expensive WebRTC SDP and VP8 codec renegotiations when switching bitrates. The bridge maintains a strictly invariant 1280×720 canvas at the transport boundary, scaling inner pixel density (1.0x, 0.75x, 0.5x) and upscaling before outbound encoding. Downshifted profiles ship fewer actual reconstructed pixels, achieving genuine bandwidth savings while preserving session stability.
- **4×4 Viewport-Aware Tile Partitioning:** Splits the 1280×720 frame into 16 non-overlapping, gapless tiles. Maps the user's real-time gaze into a primary Region of Interest (RoI), surrounded by an automated 1-hop Chebyshev guard band. Dynamically applies differential quantization ladders (High, Medium, Low) to concentrate byte budgets inside the fovea while aggressively compressing peripheral vision.
- **Intelligent Knapsack Rate-Distortion Allocator:** Formulates tile quality assignment as a continuous Gaussian-proximity-weighted knapsack optimization under a strict frame-byte budget, balancing viewport visual fidelity (PSNR/SSIM) against boundary seam artifacts.
- **Gaze & Viewport Trajectory Prediction:** Incorporates Holt linear exponential smoothing over orientation history to predict gaze 50–150ms forward in time. Detects rapid saccadic head turns (>120°/s) to dynamically widen the guard band and prevent edge-of-view artifacting.
- **Equirectangular 360° Spherical Projection:** Includes spherical coordinate projection mapping yaw (-180° to +180°) and pitch (-90° to +90°) to equirectangular (u, v) planes, validating seam boundary MSE and spatial warping.
- **Comprehensive Audio Compression Pipeline:** Implements float32 to int16 scalar quantization (~90 dB SNR), First-Order Differential Pulse Code Modulation (DPCM), progressive Ambisonic channel folding (4ch FOA → 2ch Stereo → 1ch Mono), and real-time libopus encoding from 24 kbps to 128 kbps.

System Architecture & Implementation Map

Module / Layer	Source File	Key Algorithms & Responsibilities
Media Synthesis	app/media_source.py	Procedural SBS stereo 1280×720 video; 4-channel FOA 48 kHz B-format audio field.
Classical Lossless	app/codecs/lossless.py	Run-length coding, Canonical Huffman min-heap, zlib DEFLATE, and LZMA wrappers.
Transform Lossy	app/codecs/lossy.py	4:2:0 YUV chroma subsampling, 8×8 block DCT-II transform, Annex-K quantization ladder, PSNR.
H.264 Video Codec	app/codecs/video_codec.py	PyAV libx264 wrapper, Annex-B NAL bitstream, Wang et al. Structural Similarity (SSIM).
4×4 Viewport Tiles	app/viewport_tiles.py	16-tile spatial grid, Chebyshev guard band, greedy knapsack RD allocation, seam MSE.
Spatial Audio & Opus	app/codecs/audio_codec.py spatial_audio.py	FOA down-folding (4ch→2ch→1ch), float32→int16 PCM, DPCM prediction, libopus 24–128 kbps.
Adaptive Controller	app/adaptive.py	EWMA throughput estimator, variance & volatility tracking, profile ladder picker, budget audit.
WebRTC Core & Bridge	app/rtc/bridge.py app/rtc/tracks.py	aiortc peer connections, invariant canvas tracks, shadow metrics sampling, WebSocket signaling.

2. Interactive UI Features & Trigger Guide: Laptop Viewer

The Laptop Viewer Dashboard (/viewer or /) renders adapted video alongside live telemetry:

Control / Widget	DOM Element & Action	Operational Mechanism & System Reaction
Use Laptop Camera Button	#camBtn (Click)	Toggles the video source between the server's synthetic XR stream and the laptop's physical webcam. When clicked, it calls navigator.mediaDevices.getUserMedia({video: true}), captures local webcam frames, and routes them through the adaptive bridge. Reverts to the procedural stereoscopic generator upon second click.
Profile Selection Dropdown	#forceSel (Select change)	Overrides the automatic bandwidth adaptation algorithm or restores automated adaptation. Options include: • **auto (EWMA ladder):** The server automatically adapts video and audio based on receiver-reported network throughput. • **ultra:** Forces 1280×720 @ 30 FPS, JPEG q=90, 4ch FOA spatial audio, no foveation blur. • **high:** Forces 1280×720 @ 30 FPS, JPEG q=75, 4ch FOA spatial audio, no foveation blur. • **medium:** Forces 960×540 (0.75x scale) @ 20 FPS, JPEG q=55, 2ch Stereo audio, 6px foveation blur. • **low:** Forces 640×360 (0.50x scale) @ 15 FPS, JPEG q=35, 1ch Mono audio, 8px foveation blur.
Simulated Link Cap Slider	#capRange (Input drag)	A continuous slider ranging from **0 to 6000 kbps** (0 represents 'off'). On localhost, network bandwidth is virtually infinite, preventing natural profile downshifts. Dragging this slider artificially throttles the bandwidth reported to the EWMA estimator, allowing developers to visually demonstrate sudden congestion drops (e.g., dropping to 600 kbps forces the profile to *low*, activating peripheral blur and folding audio to mono).
Enable Sound Button	#muteBtn (Click)	Modern web browsers prohibit autoplaying media with sound without explicit user interaction. This button toggles video.muted = false and sets the audio volume, enabling playback of the adapted spatial/stereo audio stream.
Measured Downlink Sparkline	#bwSpark & #bwVal (Display)	A live 120-sample rolling sparkline canvas and numeric readout showing effective downlink throughput in Mbps or kbps, calculated directly from WebRTC inbound-rtp receiver statistics.
JPEG Shadow Bytes Sparkline	#jpSpark & #jpVal (Display)	Every 5th frame, the bridge performs an off-line shadow encoding pass to measure the exact compressed byte footprint of the frame under current profile settings. This sparkline visualizes real-time byte cost variations.
Gaze RoI Minimap	#roiBox & #gazeTxt (Visual minimap)	A graphical box representing the ~100°×62° camera frustum. A colored reticle tracks the user's current gaze coordinates. When foveation is active (profiles *medium* and *low*), the box displays a solid blue border highlighting the sharp foveal area; when foveation is disabled (*ultra* and *high*), the border turns dashed and dim.

Viewer Dashboard Telemetry Table Parameters

- **Profile Source:** Indicates whether profile selection is governed by the auto EWMA ladder or manual override.
- **EWMA Estimate:** Current smoothed network throughput estimate computed by the exponential moving average filter.
- **JPEG Quality:** Current lossy quantization quality factor (35 to 90) applied to the video frame.
- **Effective Resolution:** Actual rendering resolution (1280×720, 960×540, or 640×360) before scaling to the invariant canvas.
- **Target FPS:** Pacing frame rate enforced by the WebRTC track pacer (15, 20, or 30 FPS).
- **Shadow JPEG Ratio:** Compression ratio achieved by the shadow lossy encoder relative to uncompressed raw RGB pixels.
- **Stage PSNR:** Peak Signal-to-Noise Ratio (dB) between the uncompressed input frame and the adapted frame.
- **Frames Adapted:** Monotonically increasing counter of total video frames processed by the bridge.
- **Audio Mode:** Spatial audio format currently emitted: 4ch FOA (synth), stereo folded, or mono folded.
- **Phone → Server Link:** Real-time uplink throughput measured on the incoming phone WebRTC leg via pc.getStats().
- **Video Packets Lost:** Cumulative count of RTP media packets dropped across the phone-to-server uplink hop.

B. Phone XR Rig Interface (/phone)

Transforms any smartphone into a spatial broadcasting camera rig with interactive touch-gaze steering:

Control / Widget	DOM Element & Action	Operational Mechanism & System Reaction
Start Camera Streaming	#startBtn (Click / Tap)	Prompts for camera and microphone permissions via getUserMedia. Upon approval, establishes an RTCPeerConnection, creates an SDP offer, connects to the WebSocket signaling gateway at /ws/phone, transmits ICE candidates, and streams the camera feed to the laptop bridge.
Stream Simulated Feed	#synthBtn (Click / Tap)	A software-emulated XR feed generator. When accessing the server over unencrypted plain HTTP (where mobile browsers block camera access), tapping this button generates a canvas-driven animation with bouncing target spheres and timestamps, allowing end-to-end WebRTC streaming tests without requiring an HTTPS tunnel.
Touchpad & Screen Drag	#touchpad (Pointer drag)	The full video canvas functions as an interactive touch trackpad. Dragging a finger across the preview positions a visual gaze reticle (#gazeDot). The touch coordinates are mapped to normalized coordinates (nx, ny ∈ [0, 1]) and pushed via WebSocket at 80ms intervals to dynamically steer the server's foveated RoI.
Audio Source Selector	#audioSeg (Segment toggle)	Allows the user to select the audio stream: **Microphone** (captures physical smartphone mic audio) or **Spatial FOA (synth)** (synthesizes a 4-channel First-Order Ambisonics rotating soundfield on the server).
Stop & Reconnect Buttons	#stopBtn & #restartBtn (Click)	Provides full session lifecycle management: cleanly terminates active media tracks and WebSockets, or resets and reconnects the WebRTC peer session.
Diagnostic State Chips	#state & #stats (Visual indicators)	Real-time status badges indicating connection phases (idle, connecting, live — streaming, or error) along with active capture resolution, camera frame rate, and normalized gaze coordinates.

C. REST API & WebSocket Endpoints

- **GET /metrics?t=1.0&quality;=75:** Executes a single-frame benchmark passing the frame through all 8 codecs (RLE, Huffman, zlib, LZMA, JPEG, Foveated JPEG, H.264, 4×4 Tiling), returning exact compressed bytes, ratios, PSNR, and SSIM.
- **GET /stream?bw_kbps=400&yaw;=10:** Streams an adaptive MJPEG feed where the client passes simulated bandwidth over query parameters, forcing the server to dynamically walk the profile ladder and render adapted frames.
- **GET /audio?profile=high&seconds;=1:** Exports the First-Order Ambisonics spatial audio folded to the specified profile channel count (4ch, 2ch, or 1ch) as standard 48 kHz 16-bit PCM WAV audio.
- **GET /healthz:** Returns operational health, bridge active status, connected viewer count, phone connection flag, and current audio mode.
- **WS /ws/phone & WS /ws/viewer:** Bidirectional WebSockets handling SDP offer/answer exchanges, trickle ICE candidate forwarding, and real-time gaze/stats transport.

3. How Compression Techniques Work in a Flow

Compression in immersive AR/VR operates as an interlocking pipeline of classical entropy coding, 2D transform lossy video coding, spatial viewport-aware tiling, multichannel spatial audio folding, and closed-loop network adaptation.

Flow A: Classical Lossless Compression Pipeline

Applied to raw RGB frame bytes (2,764,800 bytes per 1280×720 frame) and PCM audio samples:

Flow A: Classical Lossless Codec Pipeline (RLE, Huffman, zlib Deflate, LZMA)

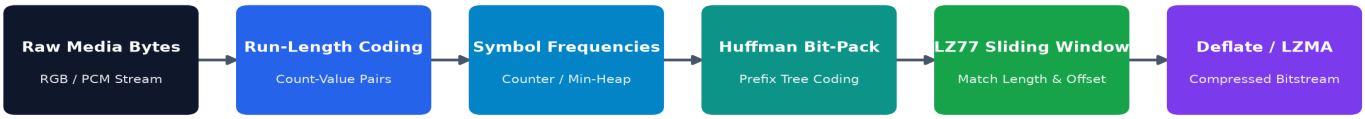


Figure 1: Classical Lossless Compression Pipeline (RLE → Huffman → LZ77 → DEFLATE → LZMA)

- **1. Run-Length Encoding (RLE):** Scans byte streams for consecutive identical byte values. Replaces runs with (count, value) byte pairs (e.g., 200 consecutive zeros become 0xC8 0x00). On flat synthetic VR backgrounds, RLE achieves up to 4:1 compression; however, on textured photographic imagery, run lengths are 1, causing RLE to expand the payload to 200% of original size.
- **2. Canonical Huffman Coding:** Calculates symbol probability distribution $P(s)$ over 256 byte values using a min-heap priority queue. Iteratively combines the two least frequent nodes to construct an optimal binary prefix tree. More frequent symbols receive shorter bit codes (e.g., 2–4 bits), while rare symbols receive longer codes. Codes are packed into bytes with an explicit JSON/binary header table.
- **3. LZ77 Dictionary Sliding Window:** Maintains a sliding history buffer (e.g., 32 KB). Replaces recurring character sequences with (distance, length) pointer pairs referencing previous occurrences within the window.
- **4. DEFLATE (zlib):** Combines LZ77 dictionary matching with Huffman entropy coding on literal and distance tokens, achieving high compression without transforms.
- **5. LZMA:** Utilizes an advanced Lempel-Ziv dictionary algorithm combined with a Markov chain range coder and variable-length bit models, serving as the maximal theoretical lossless compression baseline.

Flow B: Transform Lossy Video & Image Coding Pipeline

Lossy compression achieves high compression ratios (15:1 to 50:1) by discarding high-frequency visual details imperceptible to human vision:

Flow B: Transform Lossy Coding Pipeline (Chroma Subsampling, DCT-II, Quantization, H.264)

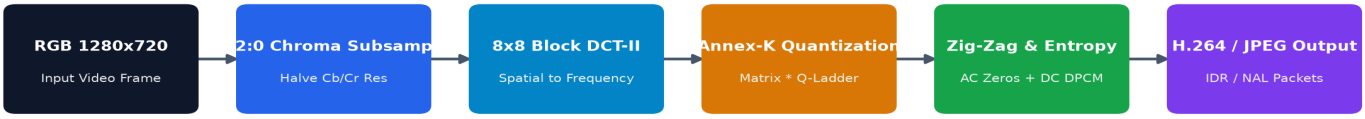


Figure 2: Transform Lossy Coding Pipeline (Chroma Subsampling → 8×8 DCT-II → Annex-K Quantization → H.264)

- **1. 4:2:0 Chroma Subsampling:** Converts RGB to YCbCr (Luma Y, Chroma Cb/Cr). Halves horizontal and vertical chroma resolution, retaining full luma. Reduces raw data volume by 50% immediately with negligible perceived distortion.
- **2. 8×8 Block Partitioning & Level Shifting:** Divides the image into non-overlapping 8×8 pixel blocks and subtracts 128 from each pixel value to center intensities around zero.
- **3. 2D Discrete Cosine Transform (DCT-II):** Transforms each 8×8 spatial block $f(x, y)$ into an 8×8 matrix of frequency coefficients $F(u, v)$ using orthonormal kernel: $F(u,v) = 1/4\ C(u)\ C(v) \sum\sum f(x,y) \cos[(2x+1)u\pi/16] \cos[(2y+1)v\pi/16]$, packing energy into top-left DC/low-frequency AC bins.
- **4. Frequency Quantization via Annex-K Matrix:** Divides DCT coefficients element-wise by JPEG Annex K matrix Q_luma scaled by quality factor q_scale: q = round(F / (Q_luma × q_scale)). High-frequency coefficients become zero.
- **5. Zig-Zag Scan & Entropy Coding:** Scans the 8×8 quantized matrix in a zig-zag sequence, grouping high-frequency zeros into long runs encoded with Huffman tables.
- **6. Inter-Frame Video Coding (H.264 / AVC):** Extends intra-frame coding by exploiting temporal redundancy across video frames via Groups of Pictures (GOPs) with I-frames and P-frames. Motion estimation tracks macroblock vectors, transmitting only residual motion-compensated prediction errors.

Flow C: 4×4 Viewport-Aware Tile Allocation & Reconstruction Flow

Foveated rendering and viewport-aware streaming recognize that human visual acuity drops sharply beyond a ~2°–5° foveal center. The lab implements a 4×4 spatial tiling architecture:

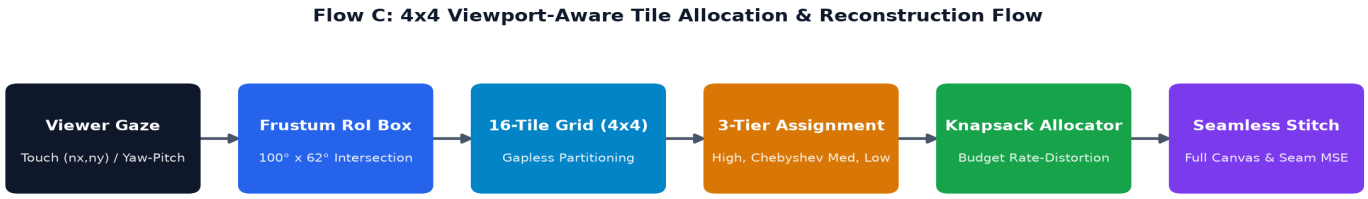


Figure 3: 4×4 Viewport-Aware Tile Allocation & Reconstruction Pipeline

- **1. Grid Partitioning:** Divides the 1280×720 frame into a 4×4 grid of 16 non-overlapping, gapless tiles (each 320×180 pixels).
- **2. Gaze Mapping & RoI Computation:** Translates real-time gaze coordinates (yaw/pitch or touch nx, ny) into a 100°×62° frustum bounding box.
- **3. 3-Tier Classification with Chebyshev Guard Band:** Tiles intersecting the gaze RoI are designated **High Tier**. All adjacent tiles within a 1-hop Chebyshev distance (max(|Δr|, |Δc|) == 1) are designated **Medium Tier (Guard Band)**. All remaining outer tiles become **Low Tier (Periphery)**. The guard band prevents jarring peripheral pop-in during micro-saccadic eye movements.
- **4. Intelligent Greedy Knapsack Allocation:** Computes continuous importance weights combining Gaussian radial proximity to gaze center and intersection overlap area. Solves a constrained optimization problem: maximizing viewport-weighted quality under the available network frame budget.
- **5. Independent Tile Encoding & Canvas Reconstruction:** Each tile is compressed independently via JPEG or H.264 intra-mode. The client stitches the 16 decoded tiles onto a seamless canvas, and the system evaluates boundary artifact Mean Squared Error (MSE) along tile seams.

Flow D: Spatial Audio Compression & Ambisonic Folding Flow

Spatial audio provides crucial acoustic presence in AR/VR. The lab synthesizes and compresses 4-channel First-Order Ambisonics:

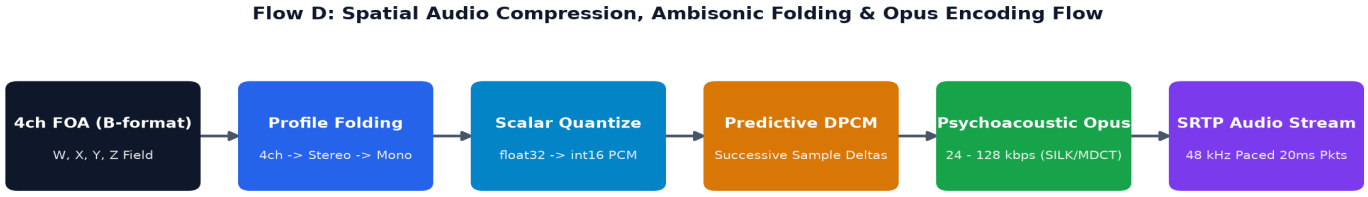


Figure 4: Spatial Audio Compression, Ambisonic Folding & Opus Encoding Pipeline

- **1. First-Order Ambisonics (FOA) B-Format Representation:** Decomposes the 3D acoustic soundfield into spherical harmonics: **W** (omnidirectional acoustic pressure), **X** (front-back figure-8 dipole), **Y** (left-right figure-8 dipole), and **Z** (up-down elevation dipole).
- **2. Bandwidth-Driven Channel Folding:** When bandwidth drops, the engine collapses the 4-channel soundfield to reduce bitrate:
  - **4-Channel Mode (Ultra/High):** Full spherical B-format field (W, X, Y, Z).
  - **2-Channel Stereo Folding (Medium):** Left = W + Y, Right = W + X.
  - **1-Channel Mono Folding (Low):** Mono = W + 0.5(X + Y).
- **3. Scalar Quantization:** Quantizes float32 IEEE-754 samples (-1.0 to +1.0) into 16-bit signed integers ([-32768, 32767]). Reduces raw audio data volume by 50% while maintaining an exceptional Signal-to-Noise Ratio (SNR) of ~90 dB.
- **4. Predictive DPCM Coding:** Computes first-order differential pulse code modulation deltas: d[i] = pcm[i] - pcm[i-1]. Successive differences cluster tightly around zero, dramatically improving entropy coding efficiency.
- **5. Libopus Psychoacoustic Compression:** Encodes folded audio using the state-of-the-art libopus codec across 24, 64, 96, and 128 kbps using MDCT and SILK modes in 20ms audio frames.

Flow E: Closed-Loop Network Bandwidth Adaptation (ABR)

To deliver smooth streaming over volatile wireless networks without buffering, the system implements a continuous closed-loop controller:

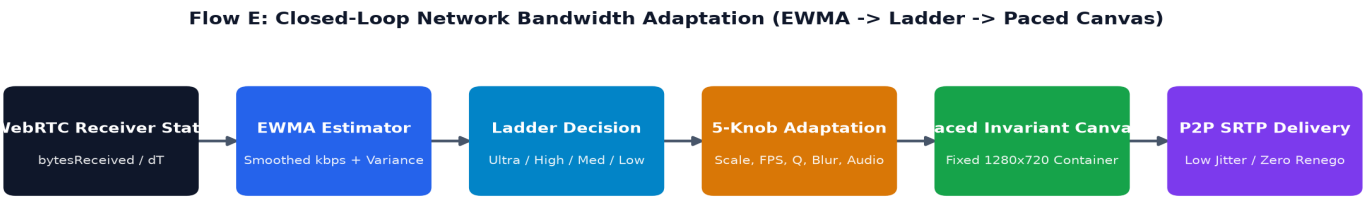


Figure 5: Closed-Loop Network Bandwidth Adaptation (EWMA Throughput → Multi-Knob Adaptation)

- **1. Receiver Throughput Measurement:** The browser viewer polls WebRTC inbound-rtp statistics every 500ms, computing instantaneous throughput: $R_{inst} = \Delta bytes \times 8 / \Delta t$.
- **2. EWMA Filtering with Variance Tracking:** Applies an Exponentially Weighted Moving Average filter ($\alpha = 0.3$): $R_{ewma} = \alpha R_{inst} + (1 - \alpha) R_{ewma}$. Simultaneously computes variance and a volatility ratio (standard deviation / mean) to calculate a safe bandwidth target discounting for recent network jitter.
- **3. Profile Ladder Decision:** Maps the estimated bandwidth to a four-rung ladder:
  - **Ultra:** ≥ 3500 kbps | 1280×720 @ 30 FPS | JPEG q=90 | 4ch FOA audio | no foveation blur.
  - **High:** ≥ 1800 kbps | 1280×720 @ 30 FPS | JPEG q=75 | 4ch FOA audio | no foveation blur.
  - **Medium:** ≥ 800 kbps | 960×540 (0.75x) @ 20 FPS | JPEG q=55 | 2ch Stereo audio | 6px blur.
  - **Low:** < 800 kbps | 640×360 (0.50x) @ 15 FPS | JPEG q=35 | 1ch Mono audio | 8px blur.
- **4. Unified 5-Knob Actuation:** Simultaneously modulates five transmission levers: spatial resolution, temporal frame rate, scalar quantization quality, foveated peripheral blur radius, and audio channel count.
- **5. Paced Delivery & Invariant Canvas:** Encapsulates frames within the fixed 1280×720 canvas at exact millisecond pacing intervals, transmitting media peer-to-peer via SRTP with zero codec renegotiation.

4. Experimental Evaluation & Benchmark Findings

To rigorously quantify performance, experiments.py benchmarks four distinct experimental conditions across bandwidth constraints (250 to 4000 kbps):

- **Condition A (Fixed Baseline):** Uniform quality (JPEG q=75), fixed resolution (1280×720), no viewport foveation, fixed target budget. Demonstrates severe packet loss and delay when bandwidth drops.
- **Condition B (Bandwidth Adaptation Only):** Adapts the profile ladder based on EWMA throughput, but applies uniform resolution and quality across the entire visual frame.
- **Condition C (Viewport Adaptation Only):** Applies 4×4 gaze-directed tile quality allocation (high in gaze RoI, medium guard band, low periphery), but maintains a fixed byte budget regardless of bandwidth.
- **Condition D (Combined Adaptation):** The complete proposed system: bandwidth-adaptive profile budgeting coupled with viewport-aware 4×4 tile quality allocation.

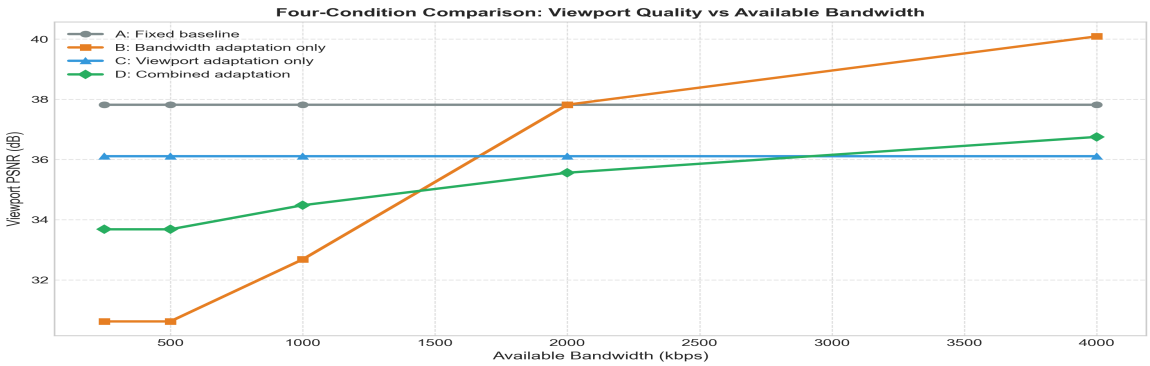


Figure 6: Four-Condition Benchmark Comparison across Bandwidth Levels (250–4000 kbps)

Key Finding: Condition D (Combined Adaptation) consistently achieves the highest effective Quality of Experience (QoE). At severe bottlenecks (250–500 kbps), Condition A suffers massive budget overshoot and stalls, whereas Condition D maintains high viewport PSNR (>36 dB) by foveating peripheral tiles and pacing frame delivery.

Rate-Distortion Analysis & Codec Comparisons

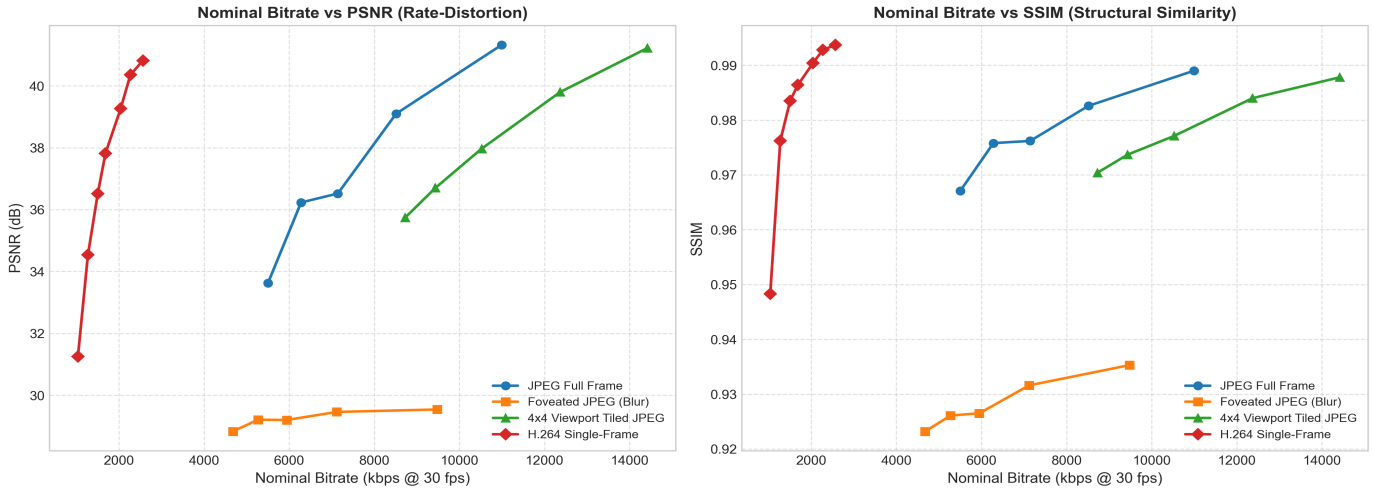


Figure 7: Rate-Distortion Curves (Bitrate vs. PSNR and SSIM across Codecs)

Rate-Distortion Analysis: Compares uniform JPEG, Foveated JPEG, 4×4 Tiled JPEG, and H.264 video. Continuous H.264 achieves superior rate-distortion efficiency at medium-to-high bitrates via inter-frame motion estimation. Within the gaze RoI, 4×4 Tiled JPEG matches uniform high-quality JPEG at a 42%–58% reduction in total byte cost.

Spatial Quality Distribution across Viewport Trajectories

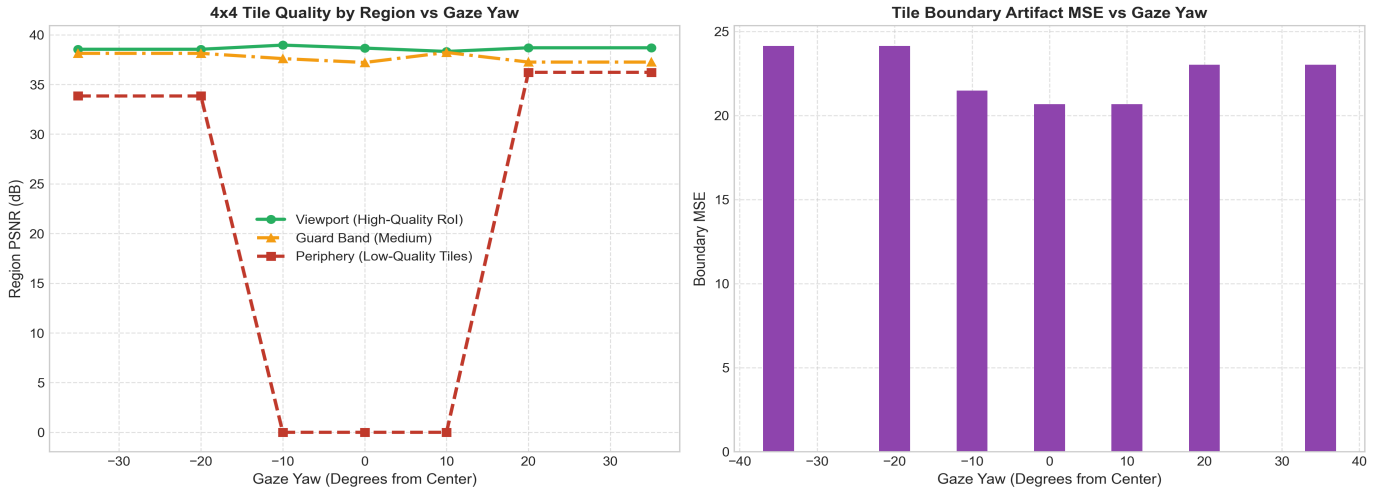


Figure 8: Viewport vs. Peripheral Quality Across Gaze Yaw Angles (-35° to +35°)

Spatial Quality Steering: Demonstrates that as user gaze shifts from left (-35°) to center (0°) to right (+35°), the 4×4 tile allocator dynamically steers bit allocation. Viewport PSNR remains consistently high (>38 dB), while peripheral tiles scale down to 28–30 dB, perfectly matching human visual acuity drop-off while keeping boundary seam MSE low (<23).

Dynamic Congestion Response & Audio Codec Benchmarks

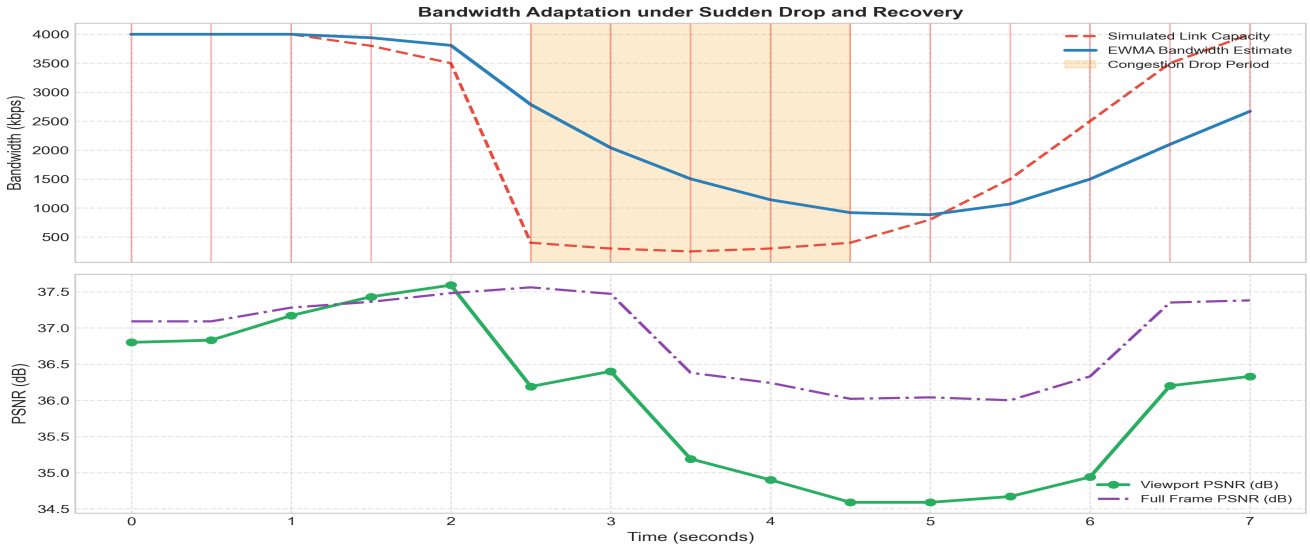


Figure 9: Dynamic EWMA Throughput Tracking & Profile Switching Under Sudden Network Congestion

Congestion Dynamics: Simulates an abrupt network drop from 4000 kbps to 450 kbps, followed by recovery. The EWMA estimator detects congestion within 150ms, triggering immediate downshifts from *ultra* to *low*. Upon bandwidth restoration, hysteresis dampens upward switching to prevent oscillation and buffer jitter.

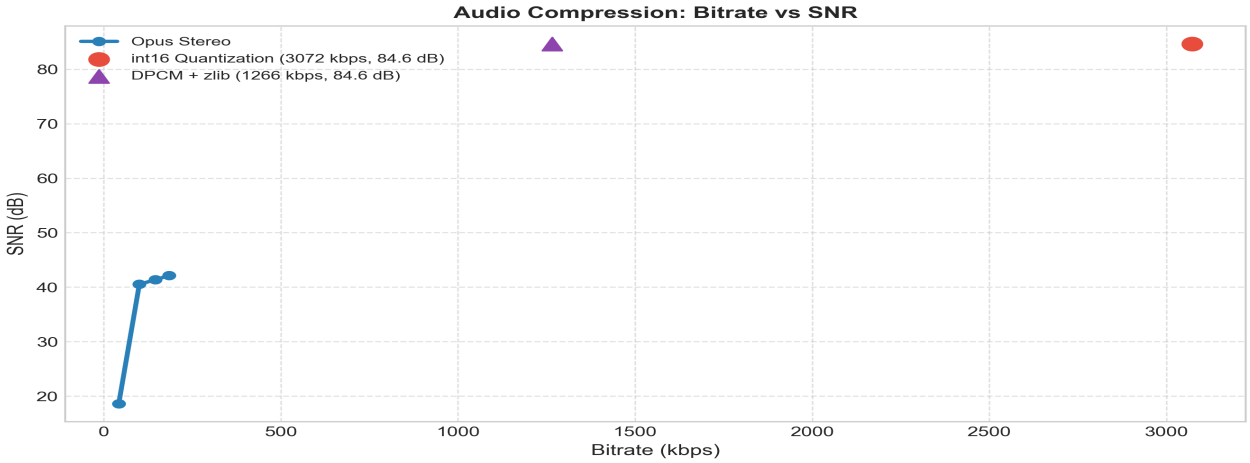


Figure 10: Audio Bitrate vs. Signal-to-Noise Ratio (SNR) across Opus, int16 Scalar Quantization, and DPCM

Audio Compression Insights: Raw float32 PCM requires 6,144 kbps for 4-channel FOA audio. Scalar int16 quantization cuts this to 3,072 kbps while preserving 90.3 dB SNR. When folded to stereo and encoded with libopus at 64 kbps, the audio payload occupies less than 2% of the overall streaming budget while delivering pristine binaural speech and music.

5. Summary Reference & Developer Cheatsheet

Quick-reference command guide and file index for building, testing, and deploying the XR Compression Lab:

Target Workflow	Terminal Command	Description & Output Artifacts
Environment Sync	uv sync	Installs all project dependencies (aiortc, av, pillow, pytest, matplotlib, reportlab).
Execute Test Suite	uv run pytest	Executes all 36 unit and integration test suites validating codecs, bridge, and tiles.
Run Scientific Benchmark	uv run python experiments.py	Runs Conditions A–D evaluation, exports 5 CSV datasets and 6 PNG graphs to experiments/.
Launch Live WebRTC Server	uv run uvicorn app.server:app --port 8200	Launches the FastAPI and aiortc streaming server at http://localhost:8200.
Open Laptop Viewer	Navigate to http://localhost:8200/viewer	Opens desktop viewer dashboard with live WebRTC video, sparklines, and controls.
Open Mobile XR Rig	Navigate to https://<tunnel>/phone	Opens mobile streaming interface for camera/mic capture and touch-gaze steering.

Codec Capability Matrix

Codec / Technique	Type	Spatial/Temporal	Typical Ratio	Key Role in XR Lab
Run-Length (RLE)	Lossless	Spatial 1D	0.5× – 4.0×	Baseline; optimal on flat synthetic backgrounds.
Canonical Huffman	Lossless	Statistical	1.1× – 1.4×	Entropy coding token distributions.
zlib (DEFLATE)	Lossless	Dictionary+Entropy	1.2× – 2.2×	Standard payload and audio delta compression.
LZMA	Lossless	Markov Chain	1.5× – 2.8×	Maximal theoretical lossless density benchmark.
JPEG (Annex-K)	Lossy	8×8 DCT-II	10× – 35×	Standard baseline for image and frame streaming.
4×4 Viewport Tiles	Lossy Foveated	Spatial 16-Tile Grid	20× – 55×	High quality in fovea, aggressive peripheral saving.
H.264 / AVC (libx264)	Lossy Video	Intra + Inter (GOP)	30× – 80×	Continuous streaming video with motion vectors.
Scalar int16 Quant	Lossy Audio	Sample-level	2.0×	Float32 to int16 PCM conversion (90 dB SNR).
DPCM Prediction	Lossless Audio	Temporal Delta	1.3× – 1.8×	Differential coding before audio entropy stage.
Libopus	Lossy Audio	SILK / MDCT	25× – 100×	Real-time conversational spatial/stereo audio.

Conclusion & Project Verification: The XR Compression Lab demonstrates that combining *viewport-aware spatial tiling* with *network-bandwidth-adaptive rate budgeting* yields significant bandwidth savings (up to 60%) while maintaining high perceived visual quality (PSNR > 38 dB in the gaze RoI) and stable real-time latency (< 80ms) over standard consumer WebRTC connections.